Układy Cyfrowe i Systemy Wbudowane	
Kierunek *Informatyka techniczna*	**Termin** *Wtorek NP, 11:10*
Imię, nazwisko, numer albumu *Karolina Obiedzińska 273034, Bartłomiej Krupa 272982*	**Data** *21 sierpnia 2025*

Sprawozdanie 5

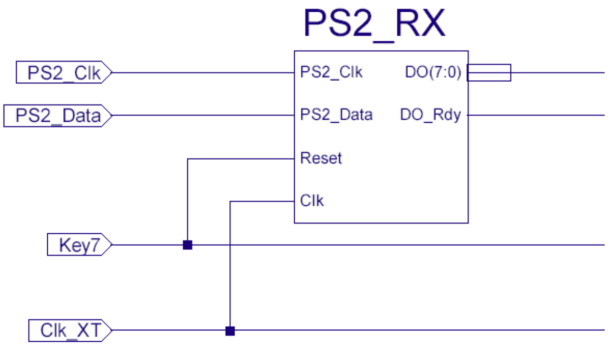
1 Wstęp

1.1 Moduł PS2_Rx

Moduł PS2_RX jest odbiornikiem danych przesyłanych z portu $PS/2$. Działa w następujący sposób:

- Odbiera bajty danych ze złącza $PS/2$.
- Sygnał $DO_Rdy =' 1'$ oznacza zakończenie odbioru pojedynczego bajtu.
- Odebrany bajt danych pojawia się na wyjściu $DO[7:0]$ i pozostaje stabilny do kolejnej transmisji.

Działanie modułu jest zsynchronizowane z sygnałem zegarowym Clk_XT. Wejścia PS2_Clk oraz PS2_Data muszą być podłączone do odpowiednich linii sygnałowych złącza PS/2.



Rysunek 1: Schemat modułu PS2_Rx

1.2 Moduł RS232_Tx

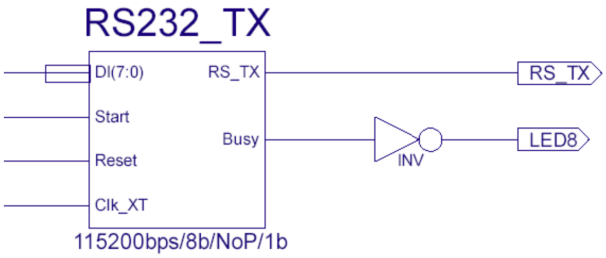
Moduł RS232_Tx odpowiada za transmisję znaków ASCII. Porty:

- DI (7:0): 8-bitowe dane wejściowe, które reprezentują kod ASCII znaku do przesłania (np. litera "o").
- Start: Sygnał startu transmisji. Gdy sygnał przyjmie wartość logiczną '1', moduł rozpoczyna przesyłanie danych. Ma to być jednotaktowy impuls.
- RS_TX: Sygnał wyjściowy, odpowiada za transmisję danych do odbiornika.

Działanie:

- Po wykryciu sygnału Start = '1', moduł przechodzi do stanu inicjalizacji transmisji.
- Kolejne 8 bitów z wejścia DI jest przesyłane na wyjście RS_TX.
- Po przesłaniu bitu stopu moduł przechodzi do stanu oczekiwania na kolejny sygnał Start.

Działanie modułu jest zsynchronizowane z sygnałem zegarowym Clk_XT, na wejście DI podajemy reprezentację znaku w systemie binarnym, u nas dla litery "o" - 01000100, według *PS/2 scan codes*.



Rysunek 2: Schemat modułu RS232_Tx

2 Zadanie 1

2.1 Opis zadania

Należało zrealizować projekt zamka szyfrowego, otwieranego za pomocą 4-literowego kodu. W naszym przypadku kod został ustalony jako "**BKKO**" – nasze inicjały, co umożliwiło poprawne przeprowadzenie późniejszych testów funkcjonalnych zamka.

Moduł zamka szyfrowego został zaprojektowany z następującymi wejściami:

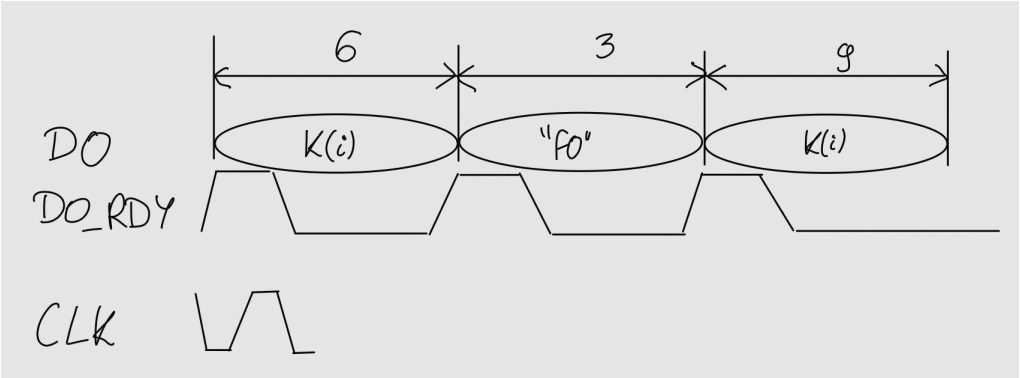
- **DO** - dane wejściowe,
- **DO_Rdy** - sygnalizacja
- **Clk** - sygnał zegarowy,
- **Reset** - sygnał resetu,

Wyjście modułu:

- **Y** - sygnał wyjściowy, gdzie $y = 1$ oznacza, że zamek został otwarty.

Z podanych materiałów: "Sygnał RS_TX należy dołączyć wprost do wyprowadzenia zewnętrznego, natomiast na wejście Clk_XT należy podać wzorcową częstotliwość 1,8432MHz."
Korzystając ze wzoru $f = \frac{1}{T}$, należy ustawić wartość "period" w pliku .ucf dla wejścia Clk_XT na: $542,5347(2)\ ns$.

Na poniższym rysunku przedstawiono ile okresów zegara ma trwać każdy etap wczytywania znaku. Biorąc pod uwagę, że sygnał DO_RDY ma być jednotaktowym impulsem trwającym jeden okres zegara wyliczono odpowiednio wartości czekania pomiędzy wczytaniami dla podanych wartości, co widać dalej na rysunku 8.

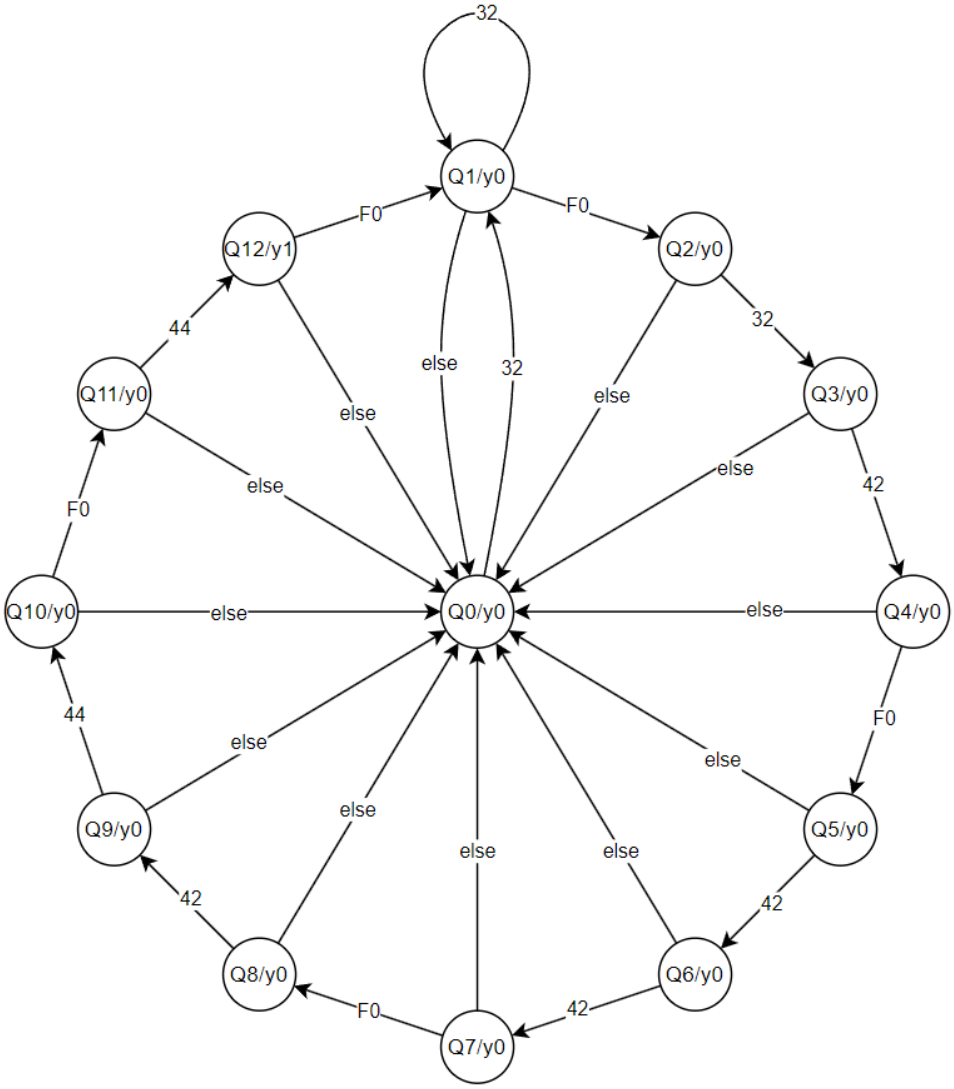


Przeprowadzono również symulację behawioralną, w której analizowano reakcję modułu na ciąg znaków wejściowych "**ABCD**ABCABCD**". Ciąg ten został wprowadzony jako wektor o długości 17 bajtów.

Symulacja miała na celu sprawdzenie poprawności działania zamka w scenariuszu, w którym kluczowe hasło "BKKO" miało być poprawnie rozpoznane w przesyłanym strumieniu danych.

2.2 Graf sekwencji

Rysunek nr 3 pokazuje automat Moore'a, z trzynastoma stanami (Q0/Y0 – Q12/Y1). Wyjście jest przypisane do każdego stanu (Y0 = 0 lub Y1 = 1), a przejścia między stanami zależą wyłącznie od sygnałów wejściowych, którymi w naszym przypadku są wartości wczytywane z klawiatury.



Rysunek 3: graf sekwencji

2.3 Program

2.3.1 Behavioral

W naszym projekcie zamka szyfrowego z 4-bitowym kodem zaproponowano 13 stanów. Liczba ta wynika z faktu, że w trakcie naciskania klawisza generowany jest jego kod, natomiast po zwolnieniu klawisza przesyłana jest wartość $F0$ oraz ponownie kod uprzednio wciśniętego klawisza. Mamy również dodatkowy stan początkowy $Q0$, ponieważ zamek został zaprojektowany jako automat Moore'a, ze względu na stabilność wyników oraz eliminację niejednoznacznych stanów przejściowych, co zapewnia niezawodne działanie układu.

```
type state_type is (Q0, Q1, Q2, Q3, Q4, Q5, Q6, Q7, Q8, Q9, Q10, Q11, Q12);
signal state : state_type := Q0;
signal next_state : state_type := Q0;
```

Rysunek 4: deklarację typu stanów oraz sygnałów

Na rysunku nr 5, 6, 7 widzimy implementację wszystkich trzech funkcji dla zamka szyfrującego. Przedstawione są na nim odpowiednio fragmenty kodu z funkcjami.
Funkcja 1: Przedstawia proces $process1$, który aktualizuje stan maszyny przy narastającym zboczu zegara CLK. Przy $RST =' 1'$, stan jest resetowany do $Q0$, w przeciwnym wypadku przechodzi do $next_state$.
Funkcja 2: Pokazuje proces $process2$, który implementuje logikę przejść między stanami maszyny w zależności od sygnałów DO i DO_Rdy. Stany Q0–Q12 zmieniają się w sposób określony przez graf przejść, bazując na wartościach sygnału DO.
Funkcja 3: Definiuje sygnał wyjściowy y. Jeśli $state = Q12$, $y =' 1'$ (zamek otwarty), w przeciwnym wypadku $y =' 0'$.

```
process1 : process (CLK)
BEGIN
   if rising_edge(CLK) then
      if RST = '1' then
         state <= Q0;
      else
         state <= next_state;
      end if;
   end if;
end process process1;
```

Rysunek 5: Funkcja 1

```
process2 : process (state, DO, DO_Rdy)
BEGIN
   next_state <= state;
   if DO_Rdy = '1' then
      case state is
         when Q0 =>
         iter <= "00";
            if DO = X"32" then
               next_state <= Q1;
            else
               next_state <= Q0;
            end if;

         when Q1 =>
         iter <= "00";
            if DO = X"F0" then
               next_state <= Q2;
            elsif DO = X"32" then

            else
               next_state <= Q0;
            end if;

         when Q2 =>
            if DO = X"32" then
               next_state <= Q3;
            else
               next_state <= Q0;
            end if;

         when Q3 =>
            if DO = X"42" then
               next_state <= Q4;
```

Rysunek 6: Funkcja 2

```
y <= '1' when state = Q12
else '0';
```

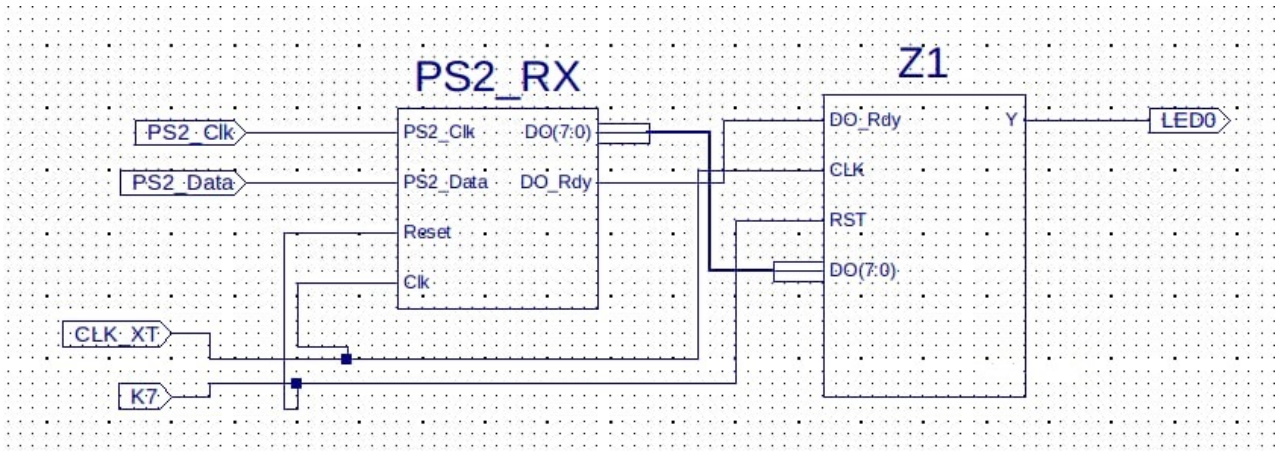
Rysunek 7: Funkcja 3

2.3.2 Schemat

Rysunek nr 8 przedstawia schemat połączenia modułu $PS2_RX$, odpowiedzialnego za odbiór danych z interfejsu PS/2, z modułem $Z1$, który realizuje logikę otwierania zamka. Moduł $PS2_RX$ odbiera dane z linii $PS2_Clk$ i $PS2_Data$. Odebrane bajty danych są udostępniane na wyjściu $DO[7:0]$, a sygnał DO_Rdy informuje o zakończeniu transmisji.

Dane oraz sygnał gotowości są następnie przekazywane do modułu $Z1$, który przetwarza je na podstawie sygnału zegarowego CLK oraz sygnału resetu RST. W wyniku działania modułu $Z1$, generowany jest sygnał wyjściowy Y, który steruje diodą LED oznaczoną jako $LED0$. Dioda gaśnie, gdy $Y =' 1'$, co oznacza, że zamek został otwarty.

Cały układ korzysta z sygnału zegarowego CLK_XT oraz sygnału resetu $K7$, które są współdzielone między moduły.



Rysunek 8: Schemat do zadania 1

2.3.3 TestBench

Kod przedstawiony na rysunku nr 9 pokazuje proces, który generuje sygnały testowe dla układu cyfrowego. Zdefiniowano typ tablicowy *key_type* oraz zmienną *input* zawierającą 17 wartości 8-bitowych (typu STD_LOGIC_VECTOR), inicjalizowanych danymi szesnastkowymi. Proces synchronizuje się ze zboczem opadającym sygnału zegarowego ($falling_edge(CLK)$) i wykonuje pętlę 'for', iterując od 16 do 0. W każdej iteracji sygnał DO_RDY jest ustawiany na $'1'$, a sygnał DO przyjmuje wartość z tablicy *input*. Po krótkim opóźnieniu ($keyTime$) DO_RDY przechodzi na $'0'$, po czym następują dodatkowe opóźnienia. Całość generuje sekwencję sygnałów zgodnie z określonymi czasami.

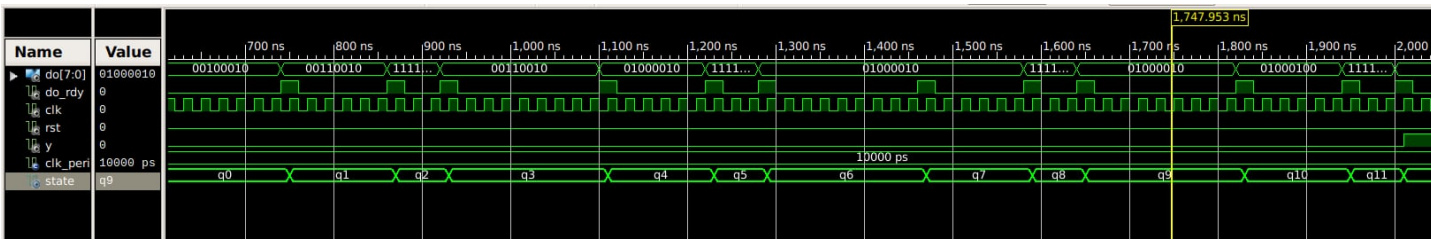
```
processl  : process
   type key_type is array(NATURAL range <>) of STD_LOGIC_VECTOR (7 DOWNTO 0);
   variable input : key_type (16 DOWNTO 0) := (
      X"2D", X"22",
      X"32", X"42", X"42", X"44",
      X"21", X"35",
      X"32", X"42", X"42", X"32", X"42", X"42", X"44",
      X"1A", X"4D"
      );

      constant keyTime : time := 2*clk_period;

   begin
         wait until falling_edge(CLK);
         for i in 16 downto 0 loop
            DO_RDY <= '1';
            Do <= input(i);
            wait for keyTime;

            DO_RDY <= '0';
            wait for keyTime*5;

            DO_RDY <= '1';
            Do <= x"F0";
            wait for keyTime;

            DO_RDY <= '0';
            wait for keyTime*2;

            DO_RDY <= '1';
            Do <= input(i);
            wait for keyTime;

            DO_RDY <= '0';
            wait for keyTime*8;

      end loop;
      wait;
   end process processl;
```

Rysunek 9: fragment testBench

2.3.4 Symulacja czasowa

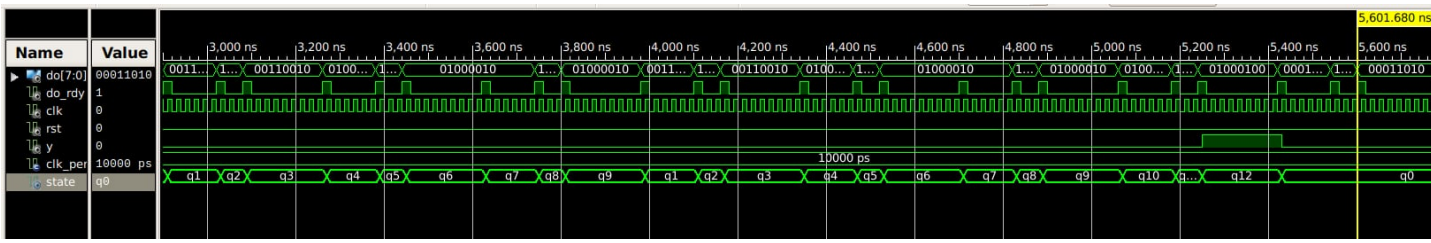
Na rysunku nr 10 przedstawiono symulację dla sekwencji wejściowej "$ABCD$", która w naszym przypadku odpowiada kodowi "$BKKO$", czyli prawidłowej konfiguracji. Z symulacji wynika, że układ przechodzi kolejno przez wszystkie 13 stanów, a w ostatnim stanie $Q12$ sygnał wyjściowy y zostaje ustawiony na wartość 1, co zgodnie z założeniami oznacza poprawne wprowadzenie kodu i otwarcie zamka.

Na rysunku nr 11 przedstawiono tę samą symulację, ale dla przypadku, w którym na wejściu zastosowano se-



Rysunek 10: Sekwencja ABCD

kwencję „$ABCABCD$”, odpowiadającą w naszym przykładzie ciągowi „$BKKBKKO$”. Można zaobserwować wzrost stanów do momentu wystąpienia prawidłowej części kodu. Jednakże przy 4. bicie dochodzi do cofnięcia się układu do stanu Q1, co widoczne jest w czasie około 4 ns. Następnie można zauważyć, że dla dalszej części sekwencji stany ponownie rosną w sposób prawidłowy, co prowadzi do poprawnego wykrycia pełnej sekwencji.



Rysunek 11: Sekwencja ABCABCD

3 Zadanie 2

3.1 Opis zadania

Zadanie polegało na rozszerzeniu projektu z Zadania 1 poprzez dodanie modułu RS232_Tx. Moduł ten miał za zadanie przesyłać literę ”o” na ekran przy użyciu programu jrs.exe, w momencie otwarcia zamka opracowanego w poprzednim etapie.

3.2 Program

3.2.1 Behavioral

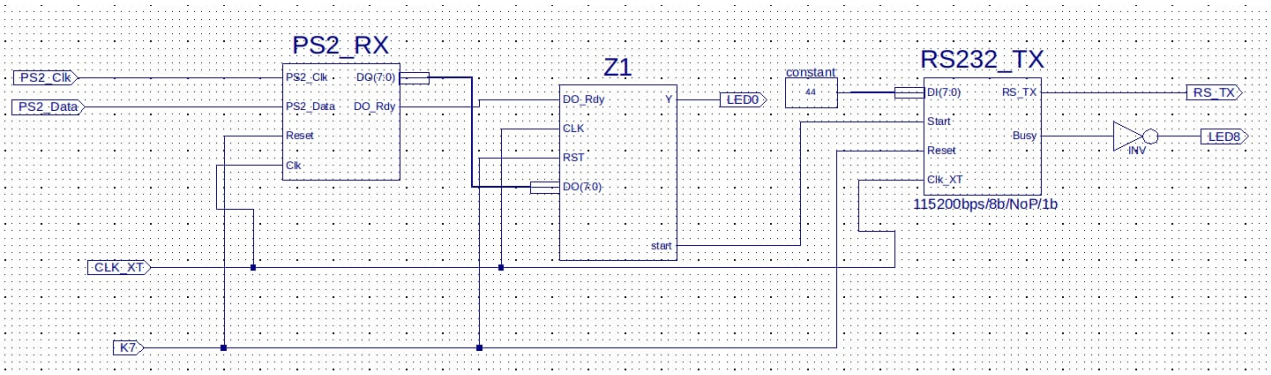
Program wzbogaciliśmy o port wyjściowy *start*, który jest podłączony do wejścia *start* modułu RS232_Tx. Oprócz tego dodaliśmy także funkcję 3 pokazaną na rysunku nr 10, która polega na ustawieniu na wyjście start sygnału ’1’ na czas jednego taktu zegarowego, jeśli ze stanu 12 mamy przejść do stanu 12, co jest równoznaczne z wpisaniem poprawnej sekwencji znaków i otwarciem zamka, po czym przekazać sygnał do modułu RS232_Tx.

```
process3: process (state, next_state, DO_Rdy)
begin
   if state=Q11 and next_state=Q12 and DO_Rdy='1' then
      start<='1';
   else
      start<='0';
   end if;
end process process3;
```

Rysunek 12: Funkcja 3

3.2.2 Schemat

Rysunek nr 13 przedstawia schemat będący rozszerzeniem rozwiązania zadania 1 o dodatkowy moduł RS232_Tx. Moduł ten ma za zadanie wyświetlić na monitorze literę „o”, której kod został przekazany do wejścia DI(7:0) w postaci stałej (constant), oraz zapalić diodę LED8 w momencie otrzymania sygnału „1” na wejściu start.



Rysunek 13: Schemat do zadania 2

4 Wnioski

W obydwu przypadkach wektorem testowych był wektor RXBKKOCYBKKBKKOZP. Obydwa układy działały poprawnie, co można było zaobserwować na wynikach symulacji behawioralnych, a także zostały poprawnie zaimplementowane sprzętowo.